



Московский государственный университет имени М.В. Ломоносова

Факультет вычислительной математики и кибернетики

Кафедра суперкомпьютеров и квантовой информатики

Отчет по заданию практикума

**“Разработка и исследование параллельных алгоритмов матрично-
векторного умножения”**

Группа: 523

Повжик Юрий Максимович

Москва, 2025

Содержание

1 Описание задания	3
2 Описание программной реализации	3
3 Описание компиляции и запуска программы.....	5
4 Результаты.....	5

1 Описание задания

Цели разработки

Разработать параллельную программу для умножения матрицы на вектор с использованием MPI. Исследовать ускорение и эффективность разработанной программы в зависимости от размера матриц и числа процессов. Построить графики времени, ускорения и эффективности. Сравнить полученное время выполнения программы с теоретической оценкой сложности разработанного алгоритма.

Задача

Необходимо реализовать параллельную программу, которая выполняет умножение большой матрицы A размером $m \times n$ на вектор x размером n и возвращает результат вектор b размером m .

2 Описание программной реализации

Программа принимает на вход 8 аргументов:

- N_x – размер матрицы по горизонтали.
- N_y – размер матрицы по вертикали.
- P_x – число процессов по горизонтали.
- P_y – число процессов по вертикали.
- `fileA` – имя файла с матрицей A .
- `fileX` – имя файла с вектором x .
- `fileB` – имя файла для вывода результата – вектора b .
- `generate_data` – опциональный аргумент. При указании 1 значения матрицы A и вектора x генерирует сама программа в соответствующих файлах. Может быть полезно для тестов и измерения времени работы алгоритма.

Результатом служит вектор b , равный произведению матрицы A на вектор x . Для умножения используется блочный алгоритм, при котором матрица A делится на примерно равные блоки между процессами. В случае если размеры входных данных не делятся нацело на число процессов, остатки от деления равномерно распределяются между процессами. Например: для распределения 11 элементов по 3 процессам каждому процессу выдается по 3 элемента и первому со вторым процессу еще по одному, получая следующее распределение: 4 4 3.

Для решения задачи были реализованы следующие функции:

- generateMatrix, generateVector – создание начальных данных для измерения времени работы программы.
- readMatrix, readVector – чтение входных данных.
- setIJ – определение границ блока каждого процесса.
- matrixVectorMultiply – умножение матрицы на вектор.
- writeResult – запись результата в файл.

В начале программы происходит инициализация входных параметров и определение размеров блоков матрицы A и вектора x для каждого из процессов функцией setIJ.

При установленном флаге generate_data – создаются начальные значения матрицы и вектора при помощи вызова generateMatrix и generateVector.

Чтение матрицы и вектора из файлов реализовано в функциях readMatrix и readVector, принимающих ссылку на матрицу или вектор, название файла, ссылки на переменные, хранящие число строк и столбцов или длину вектора и начало блока данных для соответствующего процесса.

Умножение матрицы на вектор и получение результата происходит в функции matrixVectorMultiply. Следом выполняется запись полученного результата в файл, реализованная в функции writeResult, вывод времени выполнения этапов чтения, умножения и записи нулевым процессом и освобождение ресурсов.

Оценка сложности алгоритма:

- Общее число процессов $P = P_x * P_y$
- Каждый процесс обрабатывает блок матрицы размером $N_x / P_x * N_y / P_y$
- Для получения результата процессы отправляют сообщение размером N_x / P_x и суммируют редукцией при помощи дерева глубиной $\log P_y$.

Итого: $O(\frac{N_x * N_y}{P_x * P_y} + \frac{N_x}{P_x} * \log P_y)$.

Для равных P_x и P_y и $N = N_y = N_x$: $O(\frac{N^2}{P} + \frac{N}{\sqrt{P}} * \log \sqrt{P}) = O(\frac{N^2}{P})$

3 Описание компиляции и запуска программы

Программа компилируется следующим образом:

```
mpicxx -o3 mul2.cpp -o mul2
```

Пример запуска с параметрами: `mpirun -np 4 ./mul2 100 100 2 2 A.dat x.dat b.dat 1`

Для запуска на Polus используется скрипт, который необходимо ставить в очередь на выполнение при помощи команды `bsub`:

```
#!/bin/bash
#BSUB -J mul2
#BSUB -n 16
#BSUB -W 0:30
#BSUB -m "polus-c3-ib polus-c4-ib"
#BSUB -o mul_16_10000.%J.out
#BSUB -e mul_16_10000.%J.err

echo "====="
echo "Running with MPI processes = 16 10000"
echo "=====
```

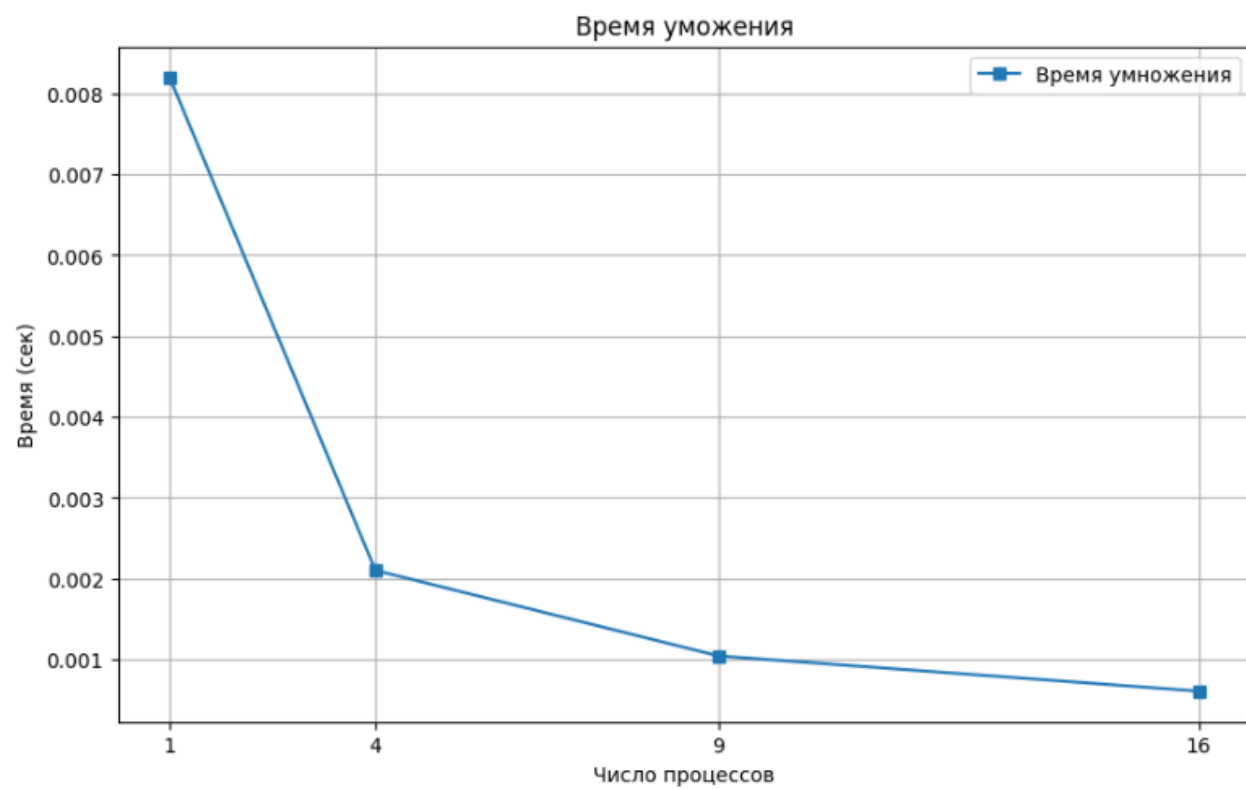
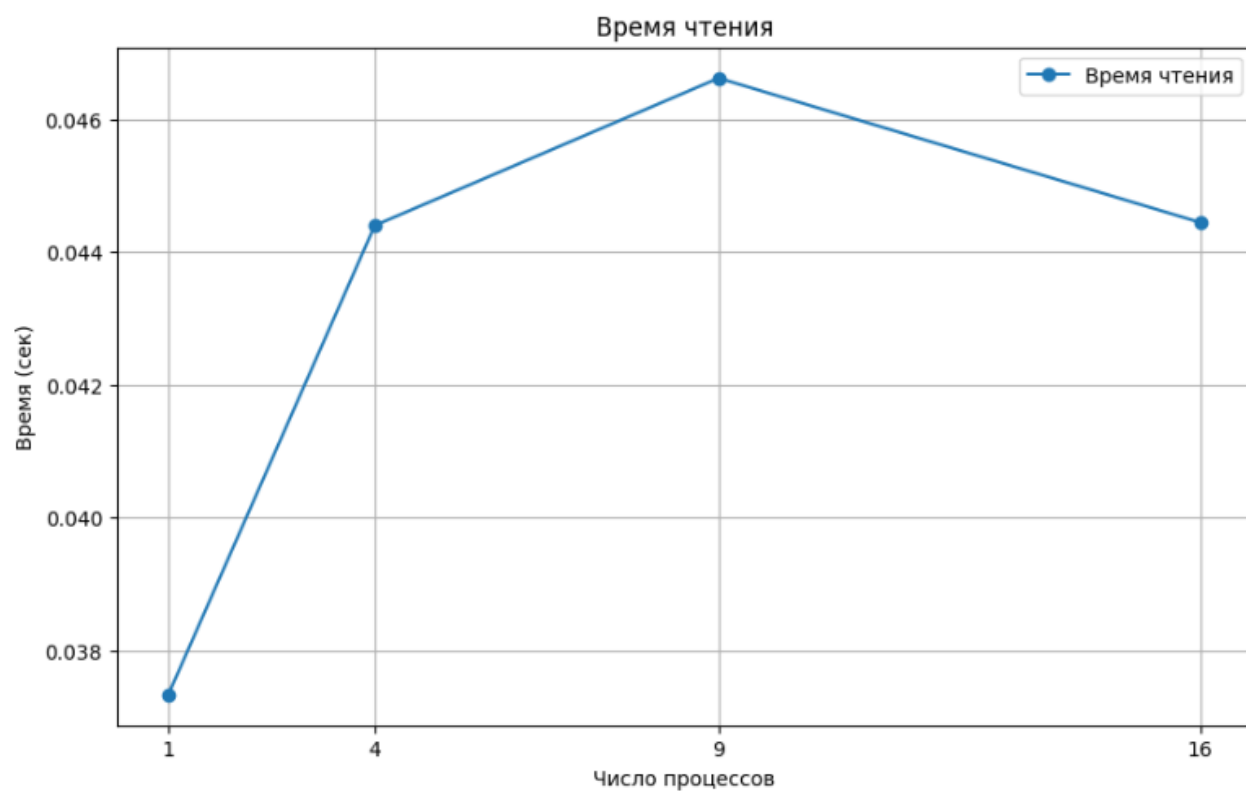
```
mpirun --map-by core --report-bindings -np 16 ./mul2 10000 10000 4 4 A.dat x.dat b.dat 1
```

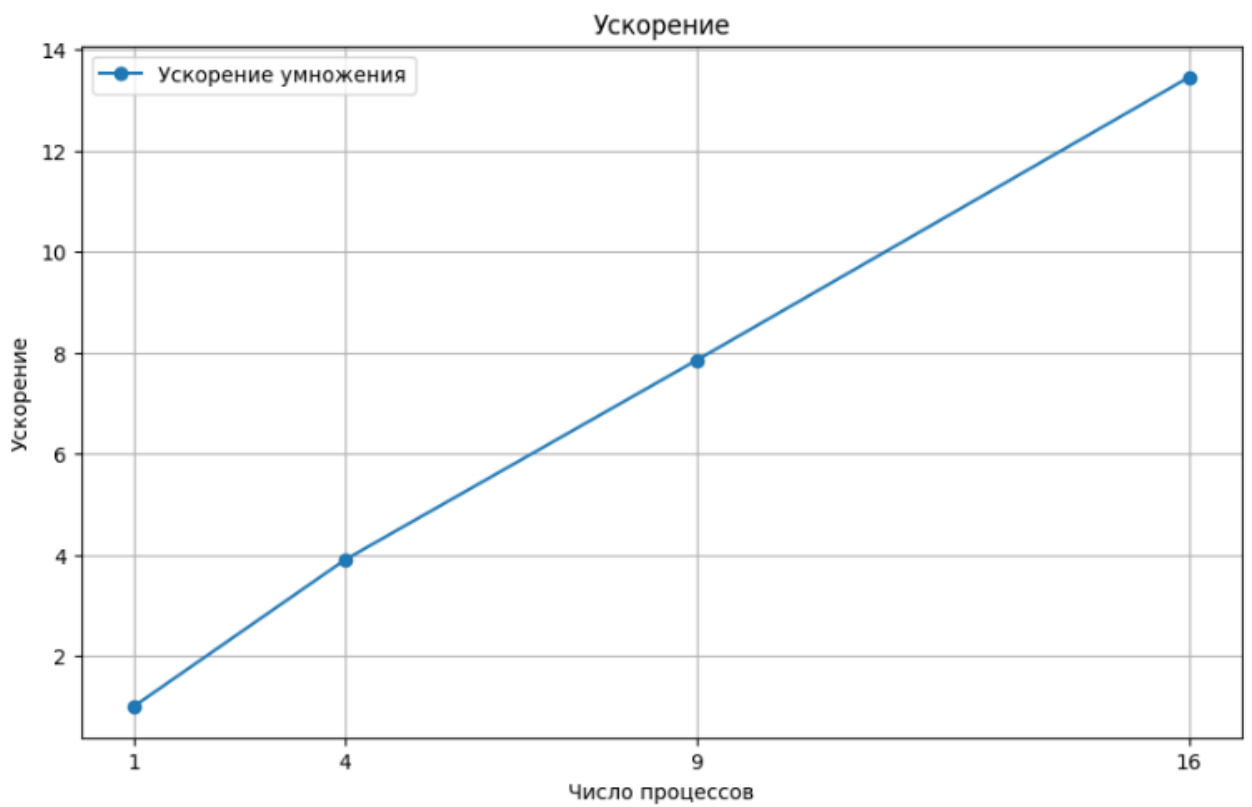
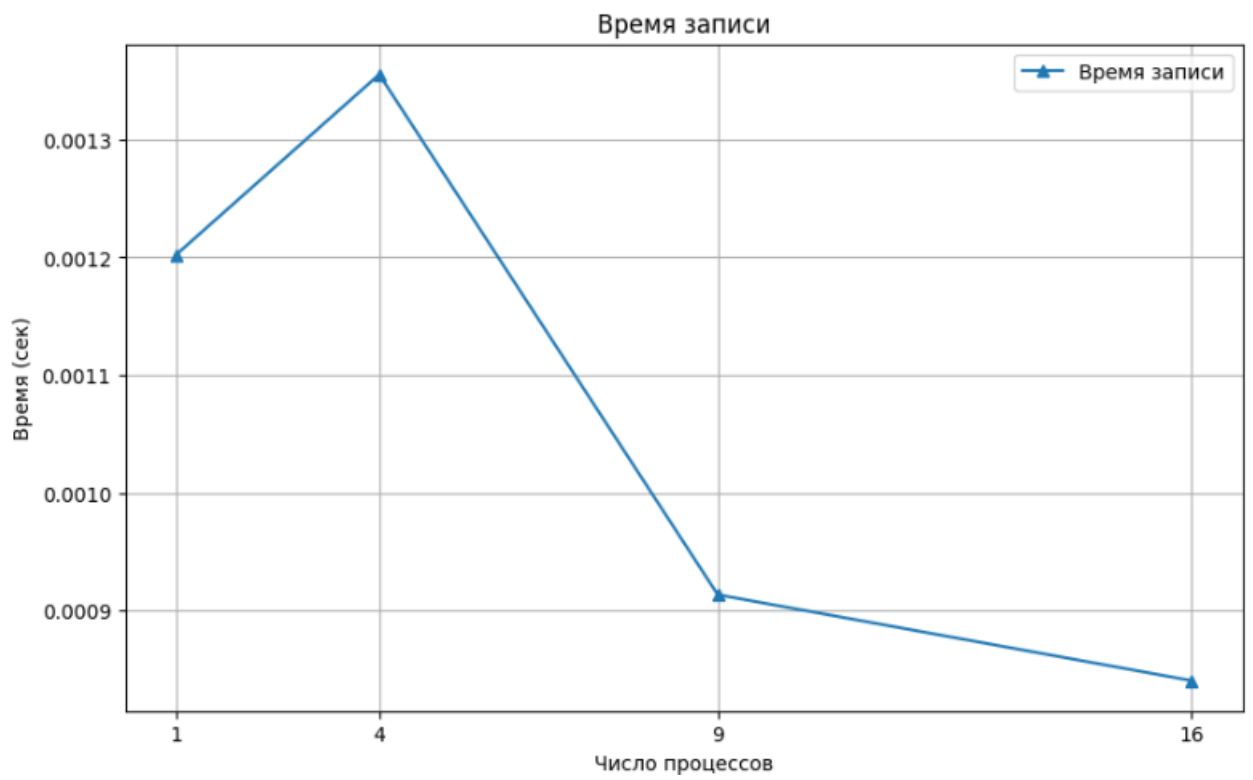
4 Результаты

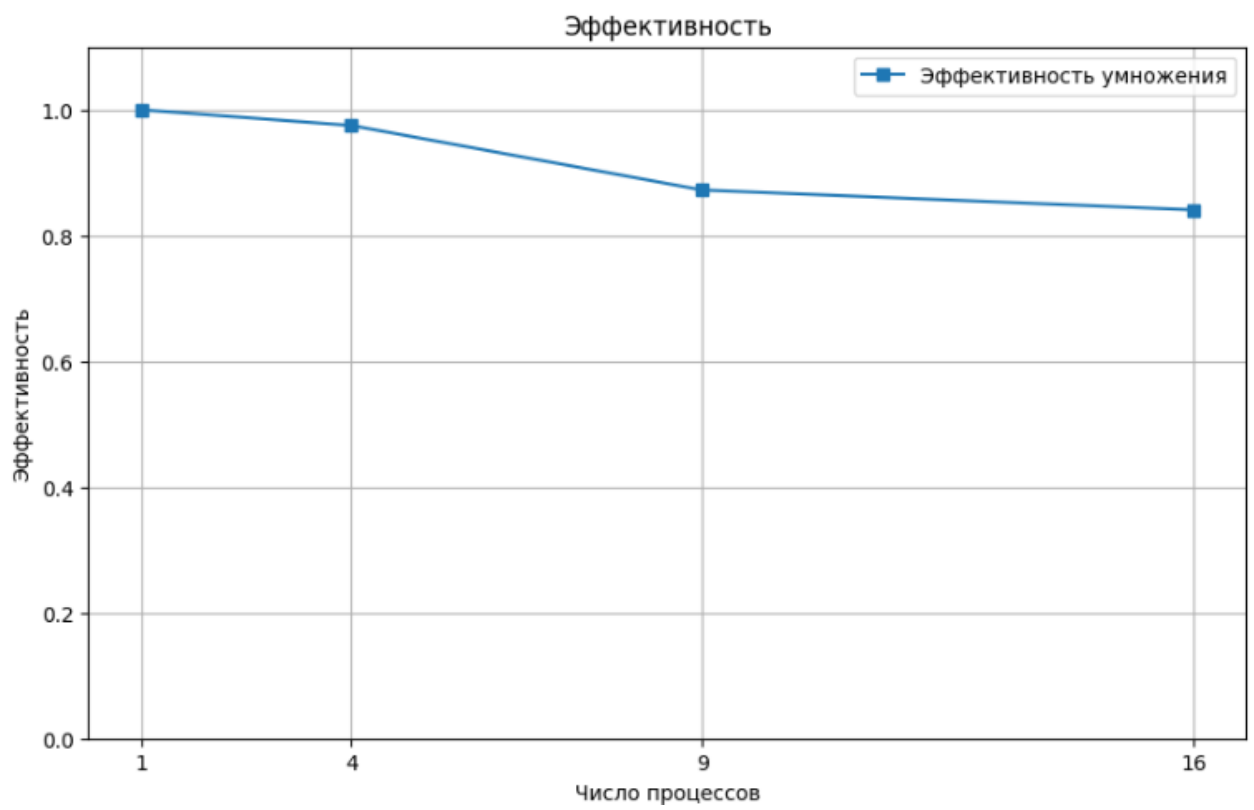
Для измерения времени работы использовались матрицы размером $N \times N$ и вектора размером N .

Ниже приведены графики времени работы операция чтения, записи и умножение, а также графики ускорения и эффективности для операции умножения:

1) $N = 1\,000$







2) $N = 10\,000$

